Introduction to Machine Learning & Scikit-learn

Building Your First ML Pipeline

Learning Objectives

- Understand the fundamentals of machine learning and its core workflow
- Set up machine learning pipelines using scikit-learn
- Implement train-test splits and understand their importance
- Apply cross-validation techniques such as k-fold and stratified k-fold
- Prepare datasets with feature scaling, encoding, and handling missing values
- Begin work on a regression project, selecting domains, datasets, and success metrics

From Data Analysis to Machine Learning

Building Intelligent Systems

What is Machine Learning?

Machine learning is a branch of artificial intelligence that enables systems to **learn patterns from data** and make predictions or decisions without being explicitly programmed. It involves using algorithms to identify relationships, extract insights, and improve performance as more data becomes available.

Key Components:

- **Task (T):** What the system needs to accomplish
- **Performance (P):** How success is measured
- **Experience (E):** Data used for learning

Types of Machine Learning

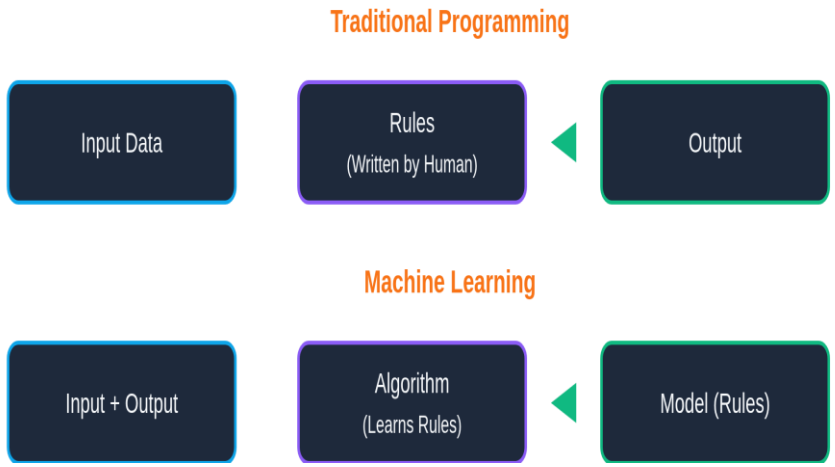
Supervised Learning – Models are trained on labeled data, learning the mapping between input features and known outputs. Common tasks include regression (predicting continuous values) and classification (predicting categories).

Unsupervised Learning – Models find patterns or structures in unlabeled data. Examples include clustering (grouping similar data points) and dimensionality reduction (reducing feature space).

Reinforcement Learning – An agent learns by interacting with an environment, receiving rewards for correct actions and penalties for incorrect ones. The model improves over time by maximizing cumulative rewards. Common tasks include game playing, robotics, and autonomous decision-making.

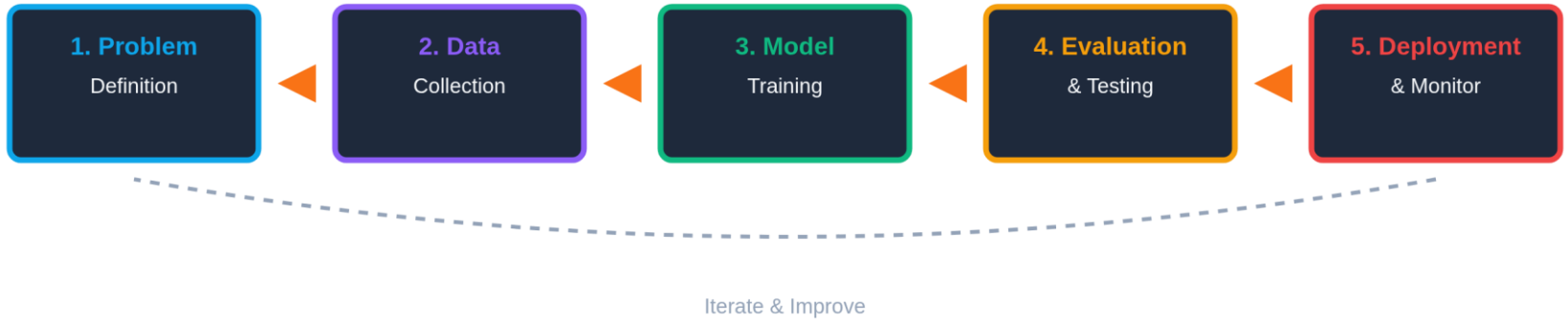
Traditional vs Machine Learning

Traditional Programming:
• Uses explicit rules written by a programmer to process input and produce output
• Logic is fixed; the system cannot improve without manual updates
• Example: A program that calculates tax based on predefined formulas
Machine Learning:
• Uses data to learn patterns and make predictions without explicit rules
• Improves performance automatically as more data becomes available
• Example: A model that predicts house prices based on historical sales data



The Machine Learning Workflow

The machine learning workflow is a structured process for developing predictive models. It begins with defining the problem and understanding objectives, followed by data collection and preprocessing. Next, features are selected and prepared, and models are trained using appropriate algorithms. The trained models are then evaluated using metrics and validation techniques, and finally, deployed for real-world use. Iteration is common, refining each step as insights are gained.



When Does ML Make Sense?

- **Complex Patterns:** Data patterns too complex for manual rules
- **Large Datasets:** Abundance of data available
- **Changing Environment:** Rules need frequent updates
- **Customization:** Output must be personalized
- **Unknown Expertise:** Human expertise doesn't exist

Introduction to Scikit-learn

Python's Premier ML Library

What is Scikit-learn?

Scikit-learn is a popular Python library for machine learning that provides simple and efficient tools for data preprocessing, model training, evaluation, and deployment. It supports a wide range of algorithms for classification, regression, clustering, and dimensionality reduction, all through a consistent and user-friendly API.

Key Features

- Consistent API
- Comprehensive algorithms
- Excellent documentation
- Active community

Built On

- NumPy
- SciPy
- Matplotlib
- Pandas-friendly

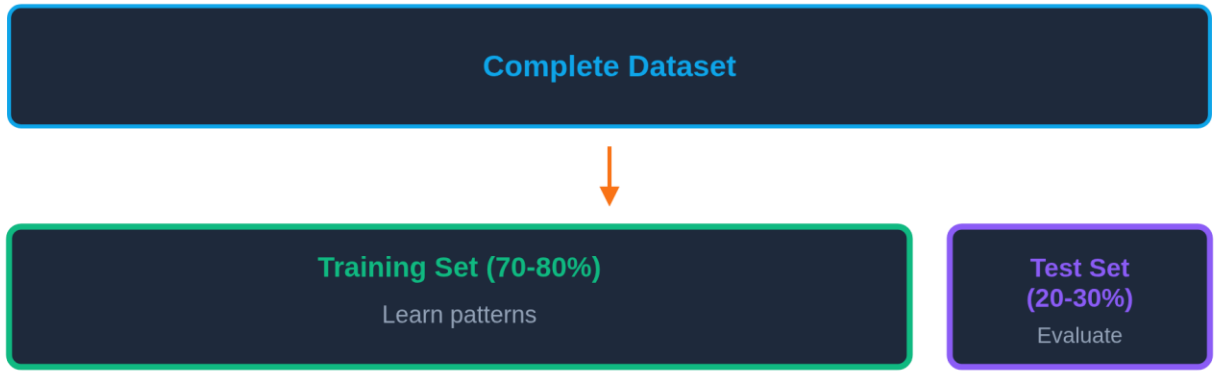
Scikit-learn API Consistency

All models follow the same interface:

```
# 1. Import & Create
from sklearn.model import Algorithm
model = Algorithm()
# 2. Train
model.fit(X_train, y_train)
# 3. Predict
predictions = model.predict(X_test)
```

Train-Test Split

Train-test split is a technique used to evaluate the performance of a machine learning model. The dataset is divided into two subsets: the training set, used to train the model, and the test set, used to assess how well the model generalizes to new, unseen data. Proper splitting helps prevent overfitting and ensures that evaluation metrics reflect real-world performance.



Never use test data for training!

Why Split the Data?

Splitting data into training and test sets ensures that a machine learning model is evaluated on unseen data, which provides a realistic measure of its performance. Without splitting, models may overfit, memorizing the training data instead of learning general patterns. Proper splitting helps in building robust, generalizable models and supports fair comparison between different algorithms.

Testing on training data = Cheating!

Model would just memorize answers

- **Avoid Overfitting:** Model memorizing vs. learning
- **Real Performance:** How model performs on unseen data
- **Generalization:** Model's ability to handle new cases

Implementing Train-Test Split

```
from sklearn.model_selection import train_test_split
# Split the data
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.2,
random_state=42
)
```

test_size: Proportion for testing (0.2 = 20%)

random_state: Reproducible splits

Introduction to Cross-Validation

Cross-validation is a technique used to assess how well a machine learning model generalizes to unseen data. Instead of a single train-test split, the dataset is partitioned into multiple folds, and the model is trained and evaluated across these folds. This provides a more reliable estimate of model performance and reduces the risk of overfitting.

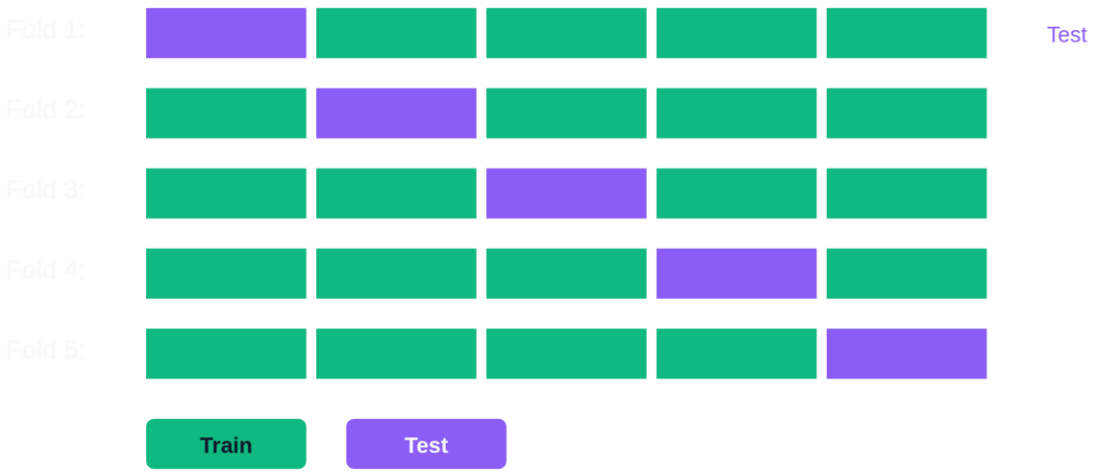
> **Problem:** Single train-test split might not be representative

Cross-Validation Solution:

- Multiple train-test splits
- More robust performance estimate
- Reduces variance in evaluation
- Better use of limited data

K-Fold Cross-Validation

K-Fold cross-validation is a method for reliably evaluating model performance. The dataset is split into k equal folds, and the model is trained k times, each time using a different fold as the test set and the remaining folds as the training set. The average performance across all folds provides a more robust estimate of the model's generalization ability.



Average scores across all folds

Implementing K-Fold CV

```
from sklearn.model_selection import cross_val_score
# Perform 5-fold cross-validation
scores = cross_val_score(
model, X, y,
cv=5
)
print(f"Average Score: {scores.mean()}")
```

Stratified K-Fold

Stratified K-Fold is a variation of K-Fold cross-validation designed for imbalanced datasets. It ensures that each fold preserves the same class distribution as the original dataset, preventing some folds from being biased toward certain classes. This leads to more reliable and consistent evaluation for classification models.

For Classification Problems:

- Maintains class distribution in each fold
- Important for imbalanced datasets
- Each fold has same proportion of classes

```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5)
```

ML Pipelines

Automating Your Workflow

What is a Pipeline?

A pipeline is a sequential workflow that automates the machine learning process, combining data preprocessing, feature transformation, and model training into a single, reproducible object. Pipelines simplify experimentation, reduce the risk of errors, and ensure that all steps are consistently applied to both training and test data.

Chain multiple steps into one workflow

One fit(), one predict()

Why Use Pipelines?

- **Convenience:** Fewer lines of code
- **Prevents Leakage:** No test data contamination
- **Reproducibility:** Same steps every time
- **Grid Search:** Tune all steps together
- **Production:** Deploy entire workflow

Creating a Simple Pipeline

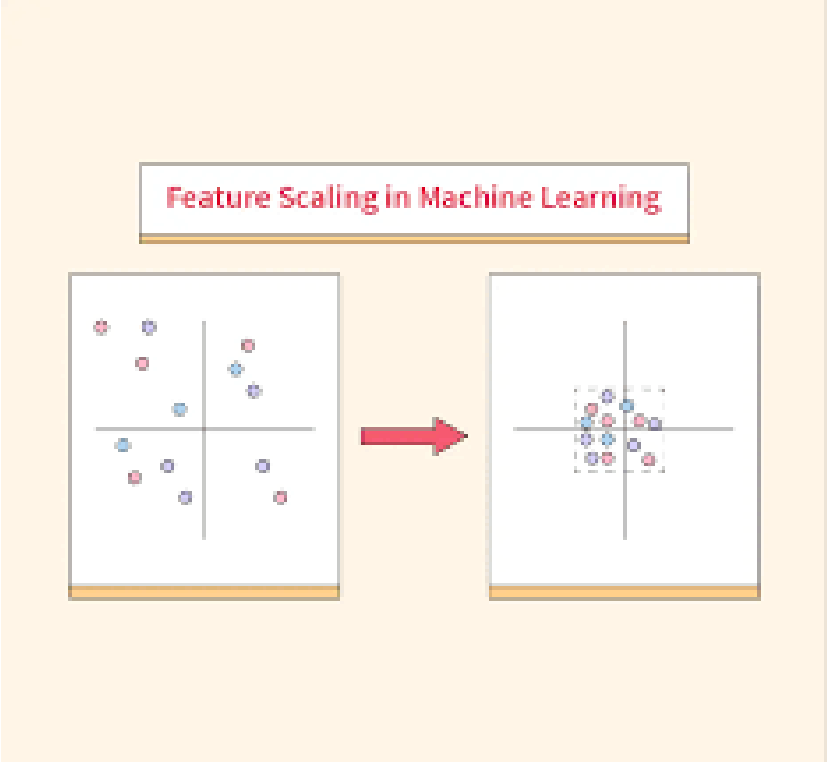
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
pipeline = Pipeline([
('scaler', StandardScaler()),
('model', LinearRegression())
])
pipeline.fit(X_train, y_train)
```

Data Preparation for ML

Making Data ML-Ready

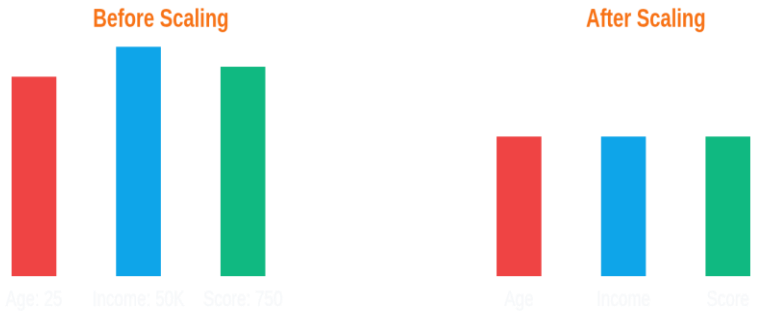
Feature Scaling

Feature scaling is the process of bringing numerical features onto a similar scale so that no single feature dominates the learning process due to its magnitude. Many machine learning algorithms rely on distance calculations or gradient optimization, making scaling essential for stable and accurate learning.
When features are on very different ranges, models may become biased toward larger-valued features, learn more slowly, or produce suboptimal results.



Why It Matters

- Improves model convergence and training speed
- Prevents features with large values from dominating the model
- Essential for distance-based models like KNN, SVM, and clustering
- Improves performance of gradient-based models such as linear and logistic regression
- Leads to more stable and interpretable models



StandardScaler

StandardScaler rescales numerical features so they have a mean of 0 and a standard deviation of 1.
This ensures that all features contribute equally to the model, especially when they are measured on different scales.
It is important for algorithms like Linear Regression, Logistic Regression, SVM, and KNN, which rely on distance or gradient calculations.
StandardScaler does not change the shape of the data distribution, it only changes the scale.

Mean = 0, Standard Deviation = 1

- Removes mean, scales to unit variance
- Sensitive to outliers
- Most common scaler

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
```

MinMaxScaler

Scale features to range [0, 1]

- Preserves original distribution shape
- Useful when you need bounded range
- Also sensitive to outliers

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X_train)
```

RobustScaler

Uses median and IQR

- Robust to outliers
- Uses median instead of mean
- Best for data with outliers

```
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X_train)
```

Encoding Categorical Variables

Machine learning models work with numbers, so categorical variables must be converted into numerical form.
Encoding allows the model to understand categories such as gender, location, or product type without adding unintended meaning.
The choice of encoding method depends on whether the categories are ordered or unordered.
Incorrect encoding can introduce bias or false relationships into the model.

OneHotEncoder

For nominal categories

Color: Red → [1,0,0]

Color: Blue → [0,1,0]

LabelEncoder

For ordinal categories

Small → 0

Medium → 1

Large → 2

OneHotEncoder Example

```
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder()
X_encoded = encoder.fit_transform(X_cat)
# Example:
# City: Lagos → [1, 0, 0]
# City: Abuja → [0, 1, 0]
# City: Kano → [0, 0, 1]
```

Handling Missing Data

Real world datasets often contain missing values due to data collection issues or incomplete records.
Missing data must be handled before training a model, since most machine learning algorithms cannot work with null values.
Common strategies include removing rows, filling values using statistical measures, or applying model based imputation.
The chosen approach should preserve data integrity and avoid introducing bias into the model.

Three main strategies:

- **Drop:** Remove rows/columns with missing values
- **Impute:** Fill with mean, median, mode
- **Predict:** Use model to predict missing values

SimpleImputer

SimpleImputer is a preprocessing tool in scikit learn used to handle missing values in a dataset.
It replaces missing values using a chosen strategy such as mean, median, most frequent value, or a constant.
This approach ensures datasets are complete before model training and works seamlessly inside pipelines.
SimpleImputer helps maintain consistency between training and testing data preprocessing.

ColumnTransformer

ColumnTransformer is a scikit learn utility used to apply different preprocessing steps to different columns in a dataset.
It is especially useful when numerical and categorical features require separate transformations such as scaling for numerical data and encoding for categorical data.
ColumnTransformer helps keep preprocessing organized, reduces errors, and integrates smoothly into machine learning pipelines.

Apply different transformers to different columns

```
from sklearn.compose import ColumnTransformer
preprocessor = ColumnTransformer([
('num', StandardScaler(), numeric_features),
('cat', OneHotEncoder(), categorical_features)
])
```

Handles mixed data types elegantly

Complete Pipeline Example

A complete machine learning pipeline combines data preprocessing and model training into a single workflow. Raw data is first cleaned, scaled, and encoded using transformers, then passed directly into a machine learning model.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
preprocessor = ColumnTransformer([
('num', StandardScaler(), num_features),
('cat', OneHotEncoder(), cat_features)
])
full_pipeline = Pipeline([
('preprocessor', preprocessor),
('model', LinearRegression())
])
```

Feature Selection

Feature selection focuses on identifying the most relevant variables that contribute to model performance. Removing irrelevant or redundant features helps reduce model complexity, improve training efficiency, and enhance generalization. Effective feature selection also improves interpretability, making it easier to understand how input variables influence predictions. Common approaches include statistical methods, model-based importance, and regularization techniques.

Feature Selection Basics

- **Remove irrelevant features**
- Improves model performance
- Reduces overfitting
- Speeds up training
- Easier model interpretation

Common methods: Correlation, Feature Importance, Variance Threshold

Summary

This session covered the foundational concepts of machine learning, including its definition, core types, and how it differs from traditional rule-based systems. The standard machine learning workflow was introduced, outlining the progression from problem formulation and data preparation to model training and evaluation.

Key components of the scikit-learn library were examined, with emphasis on its unified API and tools for reliable model development. Data splitting strategies such as train-test split and cross-validation, including K-Fold and Stratified K-Fold, were presented as essential techniques for model validation and performance assessment.

The session also addressed data preparation for machine learning, focusing on feature scaling, categorical encoding, missing data handling, and feature selection. Automated workflows using pipelines and column transformers were highlighted as best practices for building reproducible and maintainable machine learning systems.

Success Metrics

Common Regression Metrics:

- **MAE:** Mean Absolute Error (easy to interpret)
- **MSE:** Mean Squared Error (penalizes large errors)
- **RMSE:** Root Mean Squared Error (same units as target)
- **R^2 Score:** Proportion of variance explained (0-1)